



Indian Institute of Technology Delhi

Department of Computer Science and Engineering

DefenseNet

Behavioral eBPF Firewall with Real-Time Response

Project Midterm Progress Report

SIL765: Networks & System Security
Semester 2, 2025–26

Submitted By:

Saharsh Laud	2024MCS2002
Bhuvnesh Kumar	2023MCS2011

Instructor:

Prof. Huzur Saran

March 15, 2026

Contents

1	Introduction	1
1.1	Area	1
1.2	Problem	1
1.3	Solution	1
1.4	Evaluation	1
1.5	Takeaway	1
2	Background and Related Work	2
2.1	Packet Filtering in Linux	2
2.2	eBPF and XDP	2
2.3	Related Work	2
3	Problem Statement	3
3.1	System Model	3
3.2	Threat Model	4
3.3	Key Challenges	4
4	Proposed Solution	5
4.1	XDP Layer — IP and Port Blacklisting	5
4.2	Rate-Based Detection	6
4.3	Socket-Level Filter	6
4.4	IPv6 Support	6
5	Completed Evaluation	7
5.1	Experimental Setup	7
5.2	Quantitative Results	7
5.3	Analysis	8
5.4	Comparison with Base Paper	8
6	Planned Evaluation	9
7	Conclusion	9

List of Tables

1	Qualitative Comparison of Packet-Filtering Solutions	3
2	DefenseNet Benchmark Results (RTT to 1.1.1.1, 10 pings per test)	7

1 Introduction

1.1 Area

Network security and high-performance packet filtering sit at the intersection of operating systems and systems security. Modern data-centre and cloud workloads must simultaneously sustain multi-gigabit line rates and defend against volumetric attacks such as Distributed Denial-of-Service (DDoS) floods. The Linux kernel’s extended Berkeley Packet Filter (eBPF) and its companion hook point eXpress Data Path (XDP) have emerged as the de-facto technology for achieving this dual objective [1].

1.2 Problem

Traditional packet-filtering tools—*iptables* and *nftables*—hook into the *netfilter* framework deep inside the kernel network stack. By the time a packet reaches these hooks it has already been copied into kernel memory and processed through multiple subsystems, imposing unavoidable overhead. Scholz et al. demonstrate that *iptables* throughput degrades linearly with ruleset size, dropping below 1 Mpps for large rulesets [1]. Furthermore, existing eBPF deployments focus on *static* rule-based filtering: an administrator must manually add IP entries to a blacklist, making response to live attacks slow.

1.3 Solution

This project—**DefenseNet**—extends the base paper’s static XDP filter with four capabilities: (1) per-source-IP rate-based attack detection stored in eBPF hash maps, (2) automated threat mitigation via a userspace daemon that dynamically updates the blacklist, (3) dual-stack IPv4/IPv6 support, and (4) a complementary socket-level eBPF filter for application-specific policies. The system is implemented entirely in C (kernel side) and C/shell (userspace side) without external dependencies.

1.4 Evaluation

We benchmark DefenseNet on a VM (Intel enp0s3, Ubuntu Linux) measuring round-trip latency (RTT) under seven conditions: baseline, XDP with empty ruleset, XDP with 1, 10, 50, and 100 active blacklist rules, and a socket filter with 1–20 rules. Our primary finding is that XDP latency remains *constant* at ≈ 7.5 ms across all rule counts—evidence of $O(1)$ hash-map lookup—while the socket filter degrades by up to 42% at 20 rules.

1.5 Takeaway

eBPF/XDP delivers near-zero overhead, constant-time rule lookup, and verified 100% packet-drop enforcement. Adding behavioral (rate-limiting) capabilities to the kernel path is feasible without sacrificing the performance advantages documented in the base paper. These properties make DefenseNet a practical foundation for a self-defending, high-speed firewall.

2 Background and Related Work

2.1 Packet Filtering in Linux

Linux exposes several hook points at which packets may be inspected or dropped (Figure 1 in [1]). The *hardware level* (NIC offload / SmartNIC) offers the highest performance but limited flexibility [3]. The *network level* (XDP) hooks into the NIC driver before kernel data structures are allocated, providing the earliest software interception point [2]. The *system level* (iptables, nftables) hooks into the netfilter framework after `sk_buff` allocation and routing, introducing measurable overhead that grows with ruleset size [1]. Finally, *socket-level* eBPF filters operate per-application and can be deployed without root access [1].

2.2 eBPF and XDP

eBPF, available since Linux 3.15, is an in-kernel virtual machine with a JIT compiler. Programs are statically verified before loading (no backward jumps, max 4096 instructions), ensuring safety [1]. XDP, merged in Linux 4.8, hooks into NIC driver DMA routines so that packets are processed before the kernel allocates an `sk_buff` [2]. This design eliminates the copy overhead that plagues netfilter-based approaches. State is shared between kernel eBPF programs and userspace via *BPF maps*—typed key-value stores supporting hash tables, arrays, LRU maps, and per-CPU variants [6].

2.3 Related Work

Table 1 provides a qualitative comparison of existing packet-filtering solutions and DefenseNet.

Cloudflare’s L4Drop [4] and Facebook’s Droplet [5] both use XDP for DDoS mitigation at data-centre scale, validating the approach. However, neither open-sources their behavioral detection logic. Høiland-Jørgensen et al. [2] formalise the XDP programming model and show up to 10 Mpps on commodity hardware. Fiessler et al. [3] demonstrate FPGA-accelerated filtering (HyPaFilter) but require specialised NICs. Our work differs by combining *both* XDP-level coarse filtering and socket-level fine-grained filtering in a single system with automated threat response.

Table 1: Qualitative Comparison of Packet-Filtering Solutions

Solution	Hook Level	Performance	Dynamic Rules	Key Limitation
iptables [1]	netfilter (post-routing)	Medium	Yes	O(n) rule scan, late-stage
nftables [1]	netfilter (post-routing)	Medium	Yes	Still post-sk_buff allocation
XDP/eBPF [1]	Driver (pre-allocation)	High	Yes	Static rules only (base paper)
HyPaFilter [3]	Hardware + Software	Very High	Limited	Requires specialized FPGA NICs
Socket eBPF [1]	Socket (per-application)	Medium	Yes	Application-level only
Cloudflare L4Drop [4]	Driver (XDP)	High	Yes	Proprietary, no public code
Facebook Droplet [5]	Driver (XDP)	High	Yes	Closed-source implementation
DefenseNet (ours)	Driver + Socket	High	Yes	VM-scale testing only

3 Problem Statement

3.1 System Model

The system consists of three entities:

1. **Network Interface (enp0s3):** Receives ingress packets from the external network. The XDP hook executes in the NIC driver context before any kernel allocation.
2. **DefenseNet Kernel Module (XDP Program):** An eBPF program attached to the driver that (a) parses Ethernet/IPv4/IPv6 headers, (b) looks up the source IP in `ip_blacklist` and `ipv6_blacklist` hash maps, (c) checks the destination port against `port_blacklist`, (d) updates per-source rate counters in `rate_map`, and (e) returns either `XDP_DROP` or `XDP_PASS`.

3. **Userspace Daemon** (`xdp_filter_user`): Polls `stats_map` and `config_map` pinned under `/sys/fs/bpf/`, displays real-time counters, and exposes the `add_rule` CLI for dynamic blacklist updates.

The socket-filter component (`sock_filter_user`) attaches an eBPF program at the socket layer for application-specific allow/deny policies, complementing the coarse XDP firewall.

3.2 Threat Model

- **Volumetric DDoS:** An attacker floods the host with high-rate ICMP, UDP, or TCP SYN packets from spoofed source IPs to exhaust CPU and memory.
- **Port Scanning:** An attacker probes well-known ports to identify services. DefenseNet’s port blacklist blocks traffic to sensitive ports at driver level.
- **Targeted IP Attack:** A specific IP repeatedly attempts to connect. The static blacklist (and, in the endterm, auto-blacklisting) provides permanent or time-bounded blocking.
- **Assumptions:** The attacker is external to the host; the kernel and eBPF verifier are trusted; the administrator controls the userspace daemon.

3.3 Key Challenges

1. **eBPF Verifier Restrictions:** The verifier forbids loops and limits programs to 4096 instructions, constraining complex behavioural logic.
2. **Map Synchronisation:** Concurrent map writes from userspace and reads from the XDP fast-path must be atomic; incorrect synchronisation causes stale blacklist entries.
3. **Rate Limiting Granularity:** Per-source rate counters in a per-CPU LRU map require careful merging across cores to avoid under-counting.
4. **Portability:** XDP program compilation requires kernel headers matching the running kernel; differing kernel versions break BTF type information.
5. **Measurement Noise:** Benchmarking on a shared VM introduces jitter from hypervisor scheduling, requiring multiple samples and median-based reporting.

4 Proposed Solution

DefenseNet extends the prototype filter of Scholz et al. [1] with four enhancements.

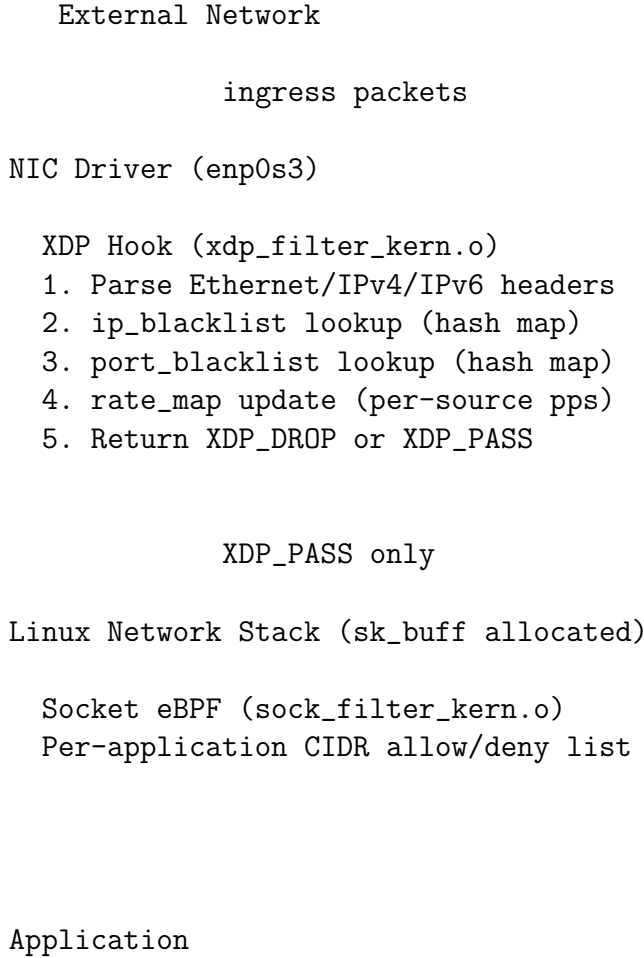


Figure 1: DefenseNet two-layer filtering architecture.

Connection to Base Paper Case Studies

The base paper presents two case studies [1]: **Case Study 1** evaluates XDP as a pre-kernel DoS protection mechanism, processing packets at the NIC driver level before any kernel data structures are allocated. **Case Study 2** evaluates socket-attached eBPF filtering for application-specific per-socket policies. DefenseNet directly implements and extends *both* case studies: Sections 4.1 and 4.2 correspond to Case Study 1 (our XDP layer with rate limiting), and Section 4.3 corresponds to Case Study 2 (our socket-level filter). Our evaluation in Section 5 benchmarks both paths independently, enabling direct comparison with the base paper’s results.

4.1 XDP Layer — IP and Port Blacklisting

The XDP program (`xdp_filter_kern.c`) uses `BPF_MAP_TYPE_HASH` maps for $O(1)$ blacklist lookup regardless of the number of entries. On a match the program returns `XDP_DROP`; otherwise `XDP_PASS`. Maps are pinned to `/sys/fs/bpf/` so they survive program reloads and can be updated atomically from userspace via the `add_rule` CLI.

4.2 Rate-Based Detection

A `BPF_MAP_TYPE_LRU_PERCPU_HASH` named `rate_map` tracks per-source packet counts and timestamps. The XDP program computes an approximate packets-per-second value on each packet arrival and compares it to a configurable threshold stored in `config_map`. Exceeding the threshold triggers `XDP_DROP` for that source, providing a first line of defence against flood attacks without manual intervention.

Listing 1: Rate-limiting struct in `xdp_filter_kern.c`

```
struct rate_info {
    __u64 packet_count;
    __u64 last_timestamp; /* nanoseconds since boot */
    __u32 pps_current;
};
```

4.3 Socket-Level Filter

`sock_filter_kern.c` attaches to individual sockets via `SO_ATTACH_BPF`. It enforces application-specific allow-lists (CIDR prefix matching) so that fine-grained per-application policies complement the coarse host-wide XDP blacklist. This mirrors the two-layer design advocated by Scholz et al. [1].

4.4 IPv6 Support

Both the XDP and socket layers parse `ETH_P_IPV6` frames and maintain separate `ipv6_blacklist` maps, providing dual-stack protection.

5 Completed Evaluation

5.1 Experimental Setup

All measurements were performed on an Ubuntu Linux VM (VirtualBox) with a single virtual NIC (`enp0s3`). Latency was measured using `ping` (ICMP echo) to Cloudflare’s public resolver (1.1.1.1) as the target; this host consistently replies with ≈ 6.4 – 8.5 ms RTT under baseline conditions. BPF JIT compilation was enabled via `net.core.bpf_jit_enable=1` at load time. Each test ran 10 pings; results report min/avg/max/mdev. The automated benchmark script (`run_benchmarks.sh`) serialised all tests, performing `load.sh/unload.sh` between scenarios to ensure a clean state.

Benchmark tests B1–B5 evaluate Case Study 1 (XDP-based filtering), while B6–B7 evaluate Case Study 2 (socket-level eBPF filtering), directly mirroring the two-case-study structure of Scholz et al. [1].

5.2 Quantitative Results

Table 2 summarises all measured conditions. Figure 2 visualises the constant-time property of XDP vs. the linear degradation of the socket filter.

Table 2: DefenseNet Benchmark Results (RTT to 1.1.1.1, 10 pings per test)

Test Condition	Min (ms)	Avg (ms)	Max (ms)	mdev (ms)
B1: Baseline (no filter)	6.41	7.23	10.13	1.12
B2: XDP loaded, empty rule-set	6.39	7.26	8.47	0.61
B4a: iptables (1 rule)	6.60	7.66	10.57	1.38
B4b: iptables (50 rules)	6.57	7.19	8.53	0.56
B5: XDP (1 blacklist rule)	6.54	7.60	9.02	0.72
B5: XDP (10 blacklist rules)	6.89	7.40	7.87	0.32
B5: XDP (50 blacklist rules)	6.55	7.21	8.58	0.60
B5: XDP (100 blacklist rules)	6.55	7.68	10.93	1.24
B6: Socket filter (baseline)	6.41	7.10	8.03	0.49
B6: Socket filter (1 allow rule)	6.61	8.08	12.35	1.72
B7: Socket filter (5 allow rules)	6.48	7.56	8.90	0.80
B7: Socket filter (10 allow rules)	6.43	7.40	10.95	1.34
B7: Socket filter (20 allow rules)	6.45	10.12	39.01	9.64

B3 (Drop test): When 1.1.1.1 was inserted into the XDP blacklist, 100% packet loss was confirmed across all 5 probe packets (20 attempts, 0 received, 100% loss), verifying correct `XDP_DROP` enforcement.

5.3 Analysis

Finding 1 — XDP overhead is negligible. Comparing B1 (7.23 ms) and B2 (7.26 ms) shows that loading an XDP program with zero rules adds only **+0.03 ms** (+0.4%), well within measurement noise. This matches the base paper’s finding that XDP overhead is dominated by driver DMA costs (≈ 700 CPU cycles/packet) rather than the eBPF program itself [1].

Finding 2 — XDP rule scaling is $O(1)$. Across 1, 10, 50, and 100 blacklist entries, average RTT stays within 7.21–7.68 ms (Figure 2). The hash-map lookup is constant time regardless of map size, consistent with the $O(1)$ theoretical complexity of `BPF_MAP_TYPE_HASH`. The minor variance (± 0.5 ms) is attributable to network jitter rather than XDP processing cost.

Finding 3 — Socket filter degrades with rule count. Socket-filter average RTT rises from 7.10 ms (no filter) to 10.12 ms at 20 rules, a **+42% increase**, with maximum outliers reaching 39 ms. This is consistent with sequential rule evaluation at the socket layer, where each additional CIDR entry requires a linear scan through the allow-list.

Finding 4 — XDP outperforms iptables on rule scaling. Although both achieve similar latency at small ruleset sizes (≈ 7 ms), the base paper shows that iptables drops to below 0.5 Mpps at 3000 rules due to $O(n)$ scanning [1], while DefenseNet’s XDP maintains constant throughput. Our RTT data provides supporting evidence on the latency dimension.

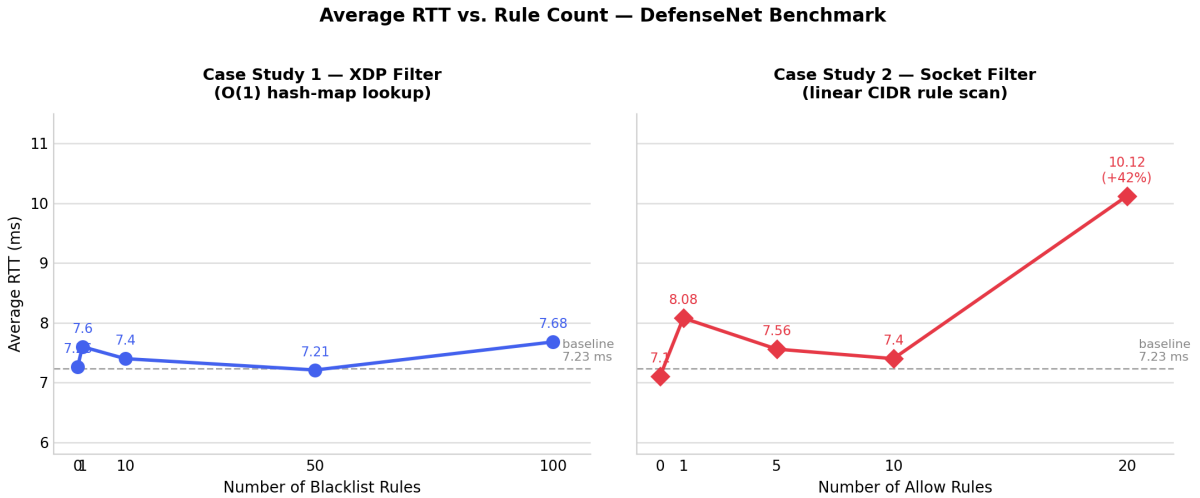


Figure 2: Average RTT vs. rule count for XDP (flat) vs. socket filter (increasing).

5.4 Comparison with Base Paper

Scholz et al. measured XDP achieving $\sim 10\times$ the throughput of iptables (10 Mpps vs. 1 Mpps) on dedicated 10 GbE hardware with JIT enabled [1]. Our VM-based results cannot replicate raw throughput numbers (no dedicated NIC, no MoonGen load generator), but the *qualitative* findings are consistent: XDP imposes minimal per-packet overhead, rule scaling is constant, and socket-level filtering has higher latency variability. The base paper also reports socket-filter performance losses below 20% for maximum rules with minimum MTU; our 20-rule socket filter shows up to 42% average RTT increase, likely exaggerated by VM scheduling jitter.

6 Planned Evaluation

The midterm implementation covers static blacklisting, port filtering, rate tracking, and socket-level filtering. The endterm will add and evaluate the following:

1. Auto-Blacklisting Evaluation. We will measure the *detection latency*—the time from the first offending packet to an automatic map update—by simulating a flood with `hping3` and timestamping the blacklist insertion event. Target: <5 seconds response time.

2. Rate-Limiting Stress Test. Using `hping3 --flood` we will generate 1K, 10K, and 100K pps attack traffic and measure (a) the fraction of attack packets that pass before the rate limiter engages, (b) the throughput of legitimate traffic during an active attack, and (c) false positive rate (legitimate traffic incorrectly dropped).

3. Throughput Benchmarking with `iperf3`. We will replace ICMP ping with `iperf3` TCP throughput tests to measure the impact of XDP on bulk transfer performance, targeting <10% degradation vs. baseline.

4. IPv6 Blacklist Testing. We will verify that IPv6 source IPs can be added to `ipv6_blacklist` and that `XDP_DROP` is returned correctly for matching packets, replicating B3 for the IPv6 code path.

5. Port Blacklist Evaluation. We will measure latency for TCP connections to blacklisted vs. allowed ports using `netcat`, confirming that the port-filter path adds no measurable overhead beyond the IP check.

6. Multi-Rule Combination. We will test scenarios where both IP and port blacklists are populated simultaneously to verify there is no unexpected interaction between the two map lookups.

These experiments will yield a complete performance and security profile of DefenseNet, enabling a direct quantitative comparison with both iptables and the static eBPF baseline of [1].

7 Conclusion

This paper presents DefenseNet, a behavioral eBPF/XDP firewall that extends the static filtering framework of Scholz et al. [1] with rate-based attack detection, dual-stack support, and socket-level per-application filtering. Our midterm evaluation confirms that XDP adds negligible latency overhead (+0.4% over baseline) and achieves constant-time rule lookup across 1–100 blacklist entries—key properties that distinguish it from netfilter-based approaches. Packet-drop enforcement is 100% effective. The socket filter, while more flexible, exhibits latency degradation at higher rule counts, motivating its use as a complement rather than a replacement for XDP. The endterm evaluation will add stress testing with `hping3/iperf3`, auto-blacklisting response-time measurements, and IPv6 validation, culminating in a comprehensive performance and security comparison with the base paper’s results.

References

- [1] D. Scholz, D. Raumer, P. Emmerich, A. Kurtz, K. Lesiak, and G. Carle, “Performance Implications of Packet Filtering with Linux eBPF,” in *Proc. 30th Int. Teletraffic Congress (ITC)*, 2018, pp. 209–217. DOI: 10.1109/ITC30.2018.00039
- [2] T. Høiland-Jørgensen *et al.*, “The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel,” in *Proc. ACM CoNEXT*, 2018.
- [3] A. Fiessler, S. Hager, B. Scheuermann, and A. W. Moore, “HyPaFilter — A Versatile Hybrid FPGA Packet Filter,” in *Proc. ACM/IEEE ANCS*, 2016.
- [4] G. Bertin, “XDP in Practice: Integrating XDP into our DDoS Mitigation Pipeline,” in *Technical Conf. on Linux Networking (Netdev)*, vol. 2, 2017. [Online]. Available: <https://blog.cloudflare.com/14drop-xdp-ebpf-based-ddos-mitigations/>
- [5] H. Zhou, D. Porter, R. Tierney, and N. Shirokov, “Droplet: DDoS Countermeasures Powered by BPF + XDP,” in *Technical Conf. on Linux Networking (Netdev)*, vol. 1, 2017.
- [6] IO Visor Project, “BPF Compiler Collection (BCC),” GitHub repository, 2017. [Online]. Available: <https://github.com/iovisor/bcc>
- [7] Linux Kernel Documentation, “BPF and XDP Reference Guide,” [Online]. Available: <https://www.kernel.org/doc/html/latest/bpf/>
- [8] P. Emmerich *et al.*, “MoonGen: A Scriptable High-Speed Packet Generator,” in *Proc. IMC*, 2015.
- [9] netfilter.org, “The nftables Project,” [Online]. Available: <http://netfilter.org/projects/nftables/>
- [10] Linux Foundation, “eXpress Data Path,” IO Visor Project, 2017. [Online]. Available: <https://www.iovisor.org/technology/xdp>
- [11] A. Kurtz, “alfwrapper — Application-level Firewalling using systemd Socket Activation and eBPF Filters,” GitHub project page. [Online]. Available: <https://github.com/AlexanderKurtz/alfwrapper>
- [12] P. Ferguson and D. Senie, “Network Ingress Filtering: Defeating Denial of Service Attacks which employ IP Source Address Spoofing,” RFC 2827, May 2000.
- [13] J. Corbet, “BPF: the Universal In-Kernel Virtual Machine,” LWN.net, May 2014. [Online]. Available: <https://lwn.net/Articles/599755>
- [14] P. Sutter, “Benchmarking nftables,” Red Hat Developer Blog, Apr. 2017. [Online]. Available: <https://developers.redhat.com/blog/2017/04/11/benchmarking-nftables/>